

Arrays

INTRODUCTION

Computer programs manipulate data stored in convenient structures known as *data structures*. Data structures play an important role in the program design. C language supports a special type of data structure known as **arrays**. It is a derived data type, the type that is built from the basic integer or float or character or a user-defined data type (*structure, union and enum*).

NEED FOR ARRAYS

Why do we need arrays? We will answer this question with the help of the following example. Let us consider a problem, where we want to input 1000 integer *numbers* and print them in reverse order, we could write a program of this form :

Example 1 :

```
#include <stdio.h>
main()
{
    int    number0;
    int    number1;
    int number2;
        .....
        .....
    int    number999;

    scanf("%d",&number0);
    scanf("%d",&number1);
    scanf("%d",&number2);
        .....
        .....
    scanf("%d",&number999);

    printf("%d\n",number999);
    printf("%d\n",number998);
    printf("%d\n",number997);
        .....
        .....
    printf("%d\n", number0);
}
```

} 1000 lines for declaring 1000 variables

} 1000 lines for reading 1000 numbers

} 1000 lines for printing 1000 numbers

This program is over 3000 lines long. If a large amount of data is to be stored, it would be a long and tedious task to think of a separate variable name for each datum, and then to type the declaration for each one of these variables.

Arrays are the solution to this problem. They are such an important part of the art of programming that they are included in every high-level language.

An array is a structured data type consisting of a group of elements that share a common name, and that are differentiated from one another by their positions within the array, and each element of the same type. A single name is given for all the 1000 numbers. The first number must be referred as ***number[0]***, the second one as ***number[1]*** and so on. The array entries are given in Figure 1.

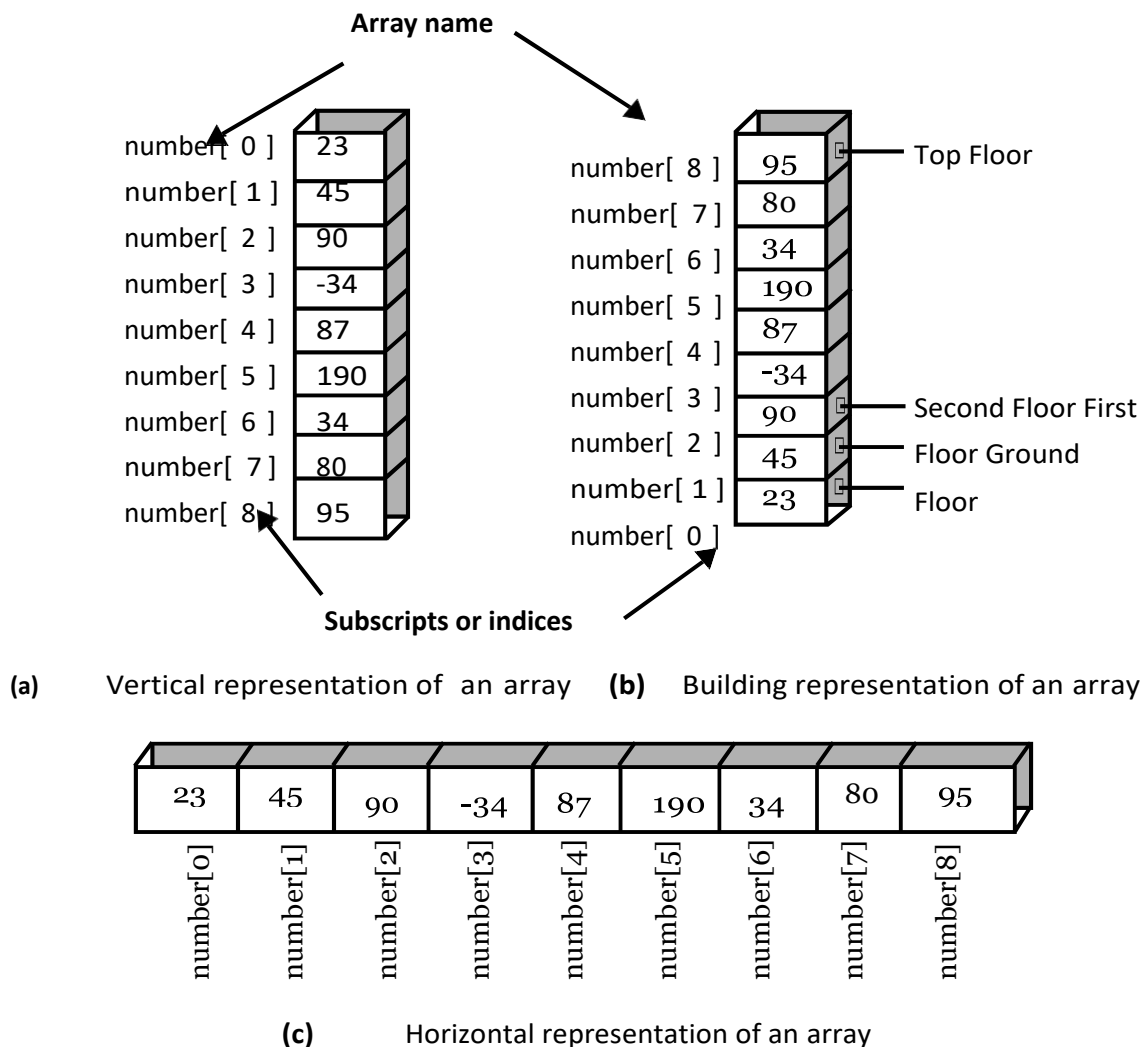


Figure 1 Array representation

The position of each of these elements can be indicated by means of a subscript. The number used to distinguish between components is called a ***subscript***, or ***index***. For example, the index of the element ***number[10]*** is 10, while the index of ***number[3]*** is 3. The index is used to access the individual element.

The subscripts of an array variable may be

1. Integer constants.
2. The subscript value cannot be negative.
3. Subscripted integer variables or integer expressions.

Example 2: Consider another problem of arranging numbers in ascending order.

Let us consider the simplest problem of arranging 3 numbers in ascending order. First number is compared with the second number if it is greater than the second then the numbers are exchanged. This is followed by comparing first and third numbers. Instead of three numbers if four numbers are to be exchanged then the above procedure we have to execute three times.

i.e . if $a > b$ then exchange a and b. if
 $a > c$ then exchange a and c. if $a >$
 d then exchange a and d.

This will bring the smallest number to be arranged to the variable a. If we are going to use ten variables a,b,c j, the program will become lengthy, and unusually complicated. Then, how do we write a program to arrange hundred numbers? Are we going to declare hundred variables? It is very difficult to write the program. In such cases, array solve our difficulties.

TYPES OF ARRAYS

Arrays are of different types -

- One-dimensional array:** A structured collection of elements, all of the same type, that is given a single name. Each element is accessed by an index that indicates the component's position within the collection.
- Two-dimensional array:** A two dimensional array is a collection of elements, all of the same type, structured in two dimensions. Each element is accessed by a pair of indices that represent the element's position in each dimension.
- Multi-dimensional array:** A collection of elements, all of the same type, ordered on N dimensions ($N \geq 1$). Each element is accessed by N indices, each of which represents the element's position within that dimension.

ONE-DIMENSIONAL ARRAY

A **one-dimensional array** is a structured collection of elements (often called array elements) that can be accessed individually by specifying the position of an element with a single index value.

Definition
One-dimensional array: A structured collection of components, all of the same type, that is given a single name. Each component (array element) is accessed by an index that indicates the component's position within the collection.

Declaration of arrays

We have seen that the type of every variable used in C program must be declared. This type declaration has to be specified for an array as well. Moreover the C compiler has to be supplied an information about the number of elements in an array (or the size of an array) and their type. This can be done by defining array type and array variables. Arrays must be declared before they are used. We declare an array by giving the type of its elements, its name, and the number of elements.

The general form of array declaration is :

Data_type Array-Name [SIZE];
--

where **Data_type** describes what is stored in each element of the array. Array elements may be of almost any type (int, float, char, double, struct, pointer or union). **SIZE** is an integer expression composed only literal or named constants. The **SIZE** specifies the number of elements in the array, must have a value greater than 0. If the value is *n*, the range of index values is **0** through *n-1*, not **1** through *n*.

Arrays are indexed start with 0, not with 1 as in many other programming languages.

For example,

Example 3: float height[5];
 int testscore[10];

The above declarations create the arrays shown in Figure 2. The **height** array has five elements, each capable of holding one *float* value. The **testscore** array has a total of ten elements, all of type *int*.

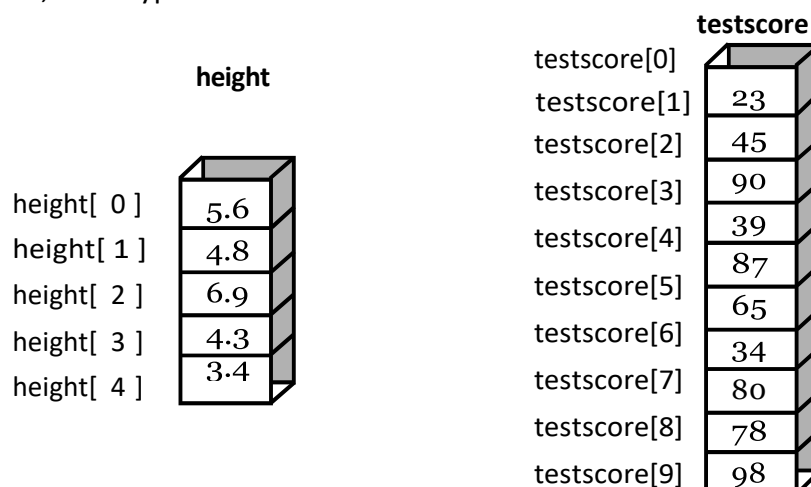


Figure 2: *height* and *testscore* arrays

Array Initialization

A special feature of arrays is that they can be initialized when they are declared. This is done by following the array name and dimension with an equal sign, followed by a pair of braces. These braces contain a series of constant values separated by commas.

Example 4 : `int     marks[5]= { 56, 67, 15, 98, 12 };`

In this declaration, *marks[0]* is initialized to 56, *marks[1]* is initialized to 67 and so on. There must be at least one initial value between the braces. If you specify too many initial values, an error message will be displayed. If you specify too few, the remaining array elements are initialized to zero.

Example 5 : `int     marks[5]= { 56 };`

In this declaration, *marks[0]* is initialized to 56, *marks[1]* to *marks[4]* are initialized to 0.

Accessing Elements of an array

Once an array is declared, individual element of an array can be accessed. To access an individual element, we write the array name, followed by an expression enclosed in square brackets. The expression specifies which element to access.

Example 6: To enter data into an array we use the following procedure.

```
int marks[6];

int index;

for( index = 0; index <=5 ; index++)
{
    printf(" Enter Marks %d ", index);
    scanf("%d",&marks[index]);
}
```

In this example, using the *for* loop, the process of input and output of **marks** is accomplished. When *index* has the value 0, the *scanf()* statement will cause the value to be stored at *marks[0]*. This process continues until index has the value greater than 5.

Example 7 : Consider the program segment given in Example 6. It has entered 6 values in the array **marks**. Now to read data from an array, we will again use *for* loop to access each element. The given program segment explains the retrieval of the values from the array.

```
for( index = 0; index <=5 ; index++)
    printf(" Subject %d Marks = %d ", index, marks[index] );
```

Program 1: A C program to read and print a one-dimensional array.

```
#include <stdio.h> main()
{
    int    a[10];          /*  max of elements   in the array is    10  */
    int    i,n;

    printf("\nEnter number of  elements in the array ");
    scanf("%d",&n);

    /*    Input array elements                */
    printf("\nEnter the array elements\n");
    for(i=0 ; i<n ; i++)
        scanf("%d",&a[i]);

    /*    printing array elements */
    printf("\nArray elements are\n");
    for(i=0 ; i<n ; i++)
        printf("\t%d",a[i]);
}
```

Program2 :A C program to find the largest element in an array and position of it's occurrence.

```
#include <stdio.h>

int main()
{
    int    a[100];
    int    largest,position,num,index;
    printf("Enter number of elements in the array ");
    scanf("%d",&num);
    for( index=0; index < num ;index++)
        scanf("%d",&a[index]);
    largest=a[0];      /*  Assign first element as  largest      */
    position = 0;      *    Position of largest element      */
    for( index = 1 ; index<num ; index++)
    {
        if(largest < a[index])
        {
            largest = a[index];
            position = index ;
        }
    }
    printf("Largest element is %d \n",largest);
    printf("Position in the array %d\n",position+1);
    return 0;
}
```

OUTPUT

Enter number of elements in the array 6
8 45 2 56 433 5
Largest element in the array is 433
Largest element's position in the array 5

Assign the first element of the array as the largest and the position as 0. Compare largest with the second element. If the second element is greater, assign the second element as the largest and the position as 1. Proceed in this way for the other consecutive elements till all elements are covered.

Program 3: Write a C program to read **N** student marks and find the class average using array.

```
#include <stdio.h>

int main()
{
    int marks[100];
    float average, sum;
    int i, n;
    printf("Enter number of students in a class ");
    scanf("%d",&n);
    sum=0;
    /*      Input Marks      */
    for(i=0 ; i<n ; i++)
    {
        scanf("%d",&marks[i]);
        sum= sum + marks[i];
    }
    average=sum/n;
    printf("\nClass average =%.2f",average);
    return 0;
}
```

OUTPUT

Enter number of students in a class: 5
57 89 47 67 60
Class average = 64.00

Program 4: Write a program to find the maximum and minimum elements in an array having **N** elements.

```
#include <stdio.h>

int main()
{
    int a[100];      /*      Array declaration      */
    int i,n,max,mini;
    printf("Enter number of elements in the array ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
```

```

        scanf("%d",&a[i] );
max =a[0]; mini=a[0];
for( i=1 ;i<n ;i++)
{
    if(max < a[i] )
        max=a[i];
    if(mini > a[i] )
        mini=a[i];
}
printf("\nMaximum element in the array is %d",max);
printf("\nMinimum element in the array is %d", mini);
return 0;
}

```

Program 5 : Write a program to generate N fibonacci numbers using array.

```

#include <stdio.h>
main()
{
    int    a[100];
    int    i , n;
    printf("Enter number of elements in the series");
    scanf("%d",&n);
    a[0]=0; /* First number in the fibonacci series */
    a[1]=1; /* Second number in the series */
    for(i=2 ; i<n ; i++)
        a[i]=a[ i - 2 ] + a[ i -1 ];
    printf("Fibonacci numbers");
    for(i=0;i<n;i++)
        printf("\t%d",a[i]);
}

```

OUTPUT

```

Enter number of elements in the series 7
Fibonacci numbers    0 1 1 2 3 5 8

```

SORTING

Sorting is an important aspect of data analysis. Sorting means arranging the array in a sequential order (ascending or descending order). Sorting will be useful to search, insert or delete a data item in an array. There are various methods of sorting :

1. Bubble sort
2. Insertion sort
3. Selection sort
4. Merge sort
5. Shell sort

6. Heapsort
7. Quick sort
8. Radix sort
9. Bucket sort
10. Address calculation sort

In this section we have given a program for the Bubble sort technique.

Bubble sort

This is one of the most widely used sorting methods.

Program 6: Program to arrange **N** numbers in the ascending and descending order using bubble sort method.

```

#include <stdio.h>
main()
{
    int    a[100];
    int    n,i,j,t;
    printf("Enter number of elements ");
    scanf("%d",&n);
    for(i=0 ; i<n ; i++)
        scanf("%d",&a[i]);
    for(i=1 ; i<n ; i++)/* Pass number */
        for(j=0 ; j<n-i ; j++)/* Comparison */
            if(a[j] > a[j+1])
            {
                t=a[j];
                a[j]=a[j+1];
                a[j+1]=t;
            }
    printf("\nList in the sorted order\n");
    for(i=0;i<n;i++)
        printf("\t%d",a[i]);
}

```

OUTPUT

```

Enter number of elements 6
18  4  56  78  45  -13
List in the sorted order
-13  4  18  45  56  78

```

SEARCHING AN ELEMENT IN THE ARRAY

This operation is used to find the location where element is available or not. In a linear search we access each element of the array and see whether it is the desired element or not. Search will be unsuccessful when all the elements are accessed and desired element is not found. In the worst case the number of comparisons is almost **N**. In best case the number of comparisons is one and in average case we may have to scan

half of the size of the array($N/2$).

Program 7 : Write a C program to find the location LOC of an ITEM in an array A which has Nelements.

```
#include<stdio.h>

main()
{
    int    a[100];
    int    n,i,item,loc=-1;

    printf("Enter number of elements");
    scanf("%d",&n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("Enter number to be searched");
    scanf("%d",&item);
    for(i=0 ; i<n ; i++)
        if(item==a[i]) /* SUCCESSFUL search*/
        {
            loc=i; break;
        }
    if(loc>=0)

        printf("\n%dis found in position%d",item,loc+1);
    else
        printf("\nItem does not exist");
}
```

TWO-DIMENSIONAL ARRAY

A one dimensional array is used to represent elements in a list or sequence of values. In many problems, however, the relationships between data items are more complex than a simple list. A two dimensional array is used to represent elements in a table with rows and columns, provided each element in the table is of the same type. An element in a two-dimensional array is accessed by specifying the row and column indices of the item in an array.

Two-dimensional arrays are called **matrices** in mathematics and **tables** in business applications; hence two-dimensional arrays are sometimes called **matrix arrays**.

Definition
Two-dimensional array: A collection of components, all of the same type and same name structured in two dimensions. Each component (array element) is accessed by a pair of indices that represent the component's position in each dimension.

There is a standard way of drawing a two dimensional array. Figure 4.3 shows a two-dimensional array with 10 rows and 10 columns. Two dimensional arrays have two subscripts. The first subscript refers to the row and the second subscript refers to the column.

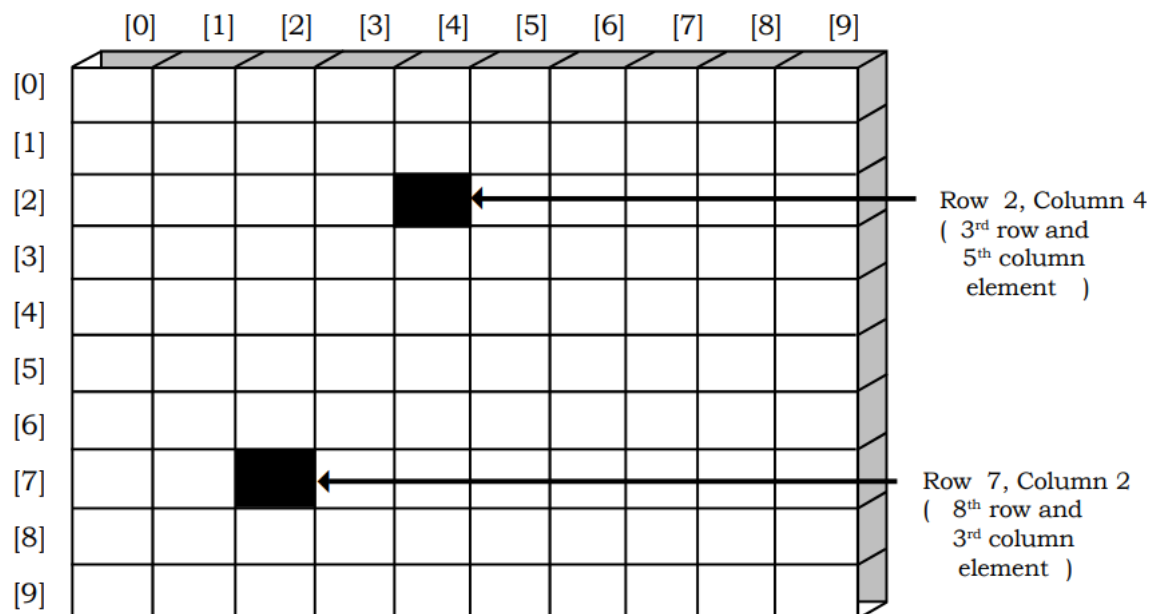


Figure 3 : A 2D-Array

Due to this representation, a two-dimensional array is also called as a **table** or a **rectangular array**. Matrices are very good examples of two-dimensional arrays.

Two-Dimensional array declaration

Two-dimensional array is declared in exactly the same way as a one-dimensional array, except that sizes must be specified for two dimensions. The general form of array declaration is:

```
Data_type array_name [ ROW SIZE ][ COLUMN SIZE ];
```

Example 8: Consider the following declaration

```
int a[10][10];
```

a is a two dimensional array which contains 10 rows and 10 columns.

Two-Dimensional array initialization

Similar to the one dimensional array, we can initialize a two-dimensional array either by initializing it in its declaration or by using assignment statements.

Example 9 :

```
int    matrix[2][3]={1,2,3,4,5,6};
```

matrix is a two dimensional array which contains 2 rows and 3 columns and these assignments would be

```
matrix[0][0] = 1    matrix[0][1] = 2    matrix[0][2] = 3
matrix[1][0] = 4    matrix[1][1] = 5    matrix[1][2] = 6
```

If the values are missing in an initializer, they are automatically set to 0.

Example 10 :

```
int matrix[2][3] ={
                    {1,1 },
                    {2 }
                    };
```

will initialize the first two elements of the first row to one, the first element of the second row to two, and all other elements to 0.

```
matrix[0][0] = 1    matrix[0][1] = 1    matrix[0][2] = 0
matrix[1][0] = 2    matrix[1][1] = 0    matrix[1][2] = 0
```

When all the elements are to be initialized to 0, the following method may be used.

Example 11:

```
int    matrix[3][4] ={{0},{0},{0}};
```

```
matrix[0][0] = 0    matrix[0][1] = 0    matrix[0][2] = 0    matrix[0][3] = 0
matrix[1][0] = 0    matrix[1][1] = 0    matrix[1][2] = 0    matrix[1][3] = 0
matrix[2][0] = 0    matrix[2][1] = 0    matrix[2][2] = 0    matrix[2][3] = 0
```

Unsize array Initialization

In array initialization, C allows to skip the size of the array. The C, then automatically creates an array big enough to hold all the initializers present. This is called **unsize array**. C automatically calculates the dimensions of unsize arrays.

Example 12 :

```
int sales[ ][4] = {    1,2,3,4
                     6,1,4,5
                     3,4,5,6
                     };
```

Only the first index can be skipped, the second index must be given.

Accessing elements of an array

Initializing an array in its declaration is impractical if the array is large. For a 100-row by 100-column array, it is very difficult to list 10,000 values. If the values are all different, input them into array at runtime.

Now we look at some examples that manipulate the components of arrays.

Example 13: Consider the following declaration:

```
int a[3][3];
```

- (i) The following **for** loops would read in 9 numbers and store them in the array **a**.

```
for( row=0 ; row < 3; row++)
    for(column=0; column<3 ; column++)
        scanf("%d",&a[row][column] );
```

- (ii) The following **for** loops would print matrix elements in the matrix format.

```
for( row=0 ; row < 3; row++)
{
    for( column=0 ; column < 3 ; column++)
        printf("\t%d",a[row][column] );
    printf("\n");
}
```

Program 8: A C program to read and display a square matrix.

```
#include<stdio.h>
main()
{
    int    a[10][10];
    int    n,i,j;
    printf("Enter order of the matrix ");
    scanf("%d",&n);
    printf("Enter matrix elements "); /*    Row major reading    */
    for(i=0 ; i<n ; i++) /*    Row number    */
        for(j=0 ; j<n ; j++) /*    Column number */
            scanf("%d",&a[i][j]);

    printf("Matrix A    is\n");
    for(i=0 ; i<n ; i++)
    {
        for(j=0 ; j<n ; j++)
            printf("%d\t",a[i][j]);
        printf("\n");
    }
}
```

Program 9: A C program to read a matrix of order N x M and display its transpose.

```
#include<stdio.h>

main()
{
    int    a[10][10],b[10][10];/*      Arraydeclaration      */
    int    n,i,j;
    printf("Enter order of the matrix ");
    scanf("%d %d",&n,&m);
    printf("Enter matrix elements");
    for(i = 0;    i<n;    i++)
        for(j = 0;    j<m;    j++)
        {
            scanf("%d",&a[i][j]);
            b[j][i]=a[i][j];
        }
    printf("Matrix A\n");
    for(i=0; i<n;    i++)
    {
        for(j = 0; j<m; j++)
            printf("%d\t",a[i][j]);
        printf("\n");
    }
    printf("Transpose of A\n");
    for(i=0;    i<m;    i++)
    {
        for(j=0    ;    j<n;    j++)
            printf("%d\t",b[i][j] );
        printf("\n");
    }
}
```

Program 10: Input two N x N matrices A and B, display the sum of A and B.

```
#include<stdio.h>

main()
{
    int a[10][10],b[10][10],c[10][10],i,j,n;

    printf("Enter order of the matrix");
    scanf("%d",&n);
    printf("\nInput A matrix elements\n");
    for(i=0; i<n;i++)
        for(j=0;j<n; j++)
            scanf("%d",&a[i][j]);

    printf("\nInput B matrix elements\n");
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d",&b[i][j]);

    for(i=0; i<n;i++)
        for(j=0; j<n; j++)
            c[i][j]=a[i][j]+b[i][j];

    printf("\nSum matrix\n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
            printf("%d\t",c[i][j]);
        printf("\n");
    }
}
```

Program 11: Input two NxN matrices A and B, display the product of A and B.

```
#include<stdio.h>
main()
{
    int    a[10][10],b[10][10],c[10][10],i,j,n;
    printf("Enter order of the matrix");
    scanf("%d",&n);
    printf("\nInput A matrix elements\n");
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d",&a[i][j]);

    printf("\nInput B matrix elements\n");
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
            scanf("%d",&b[i][j]);

    /*      Find the product of two matrices      */
    for(i=0; i<n; i++)
        for(j=0; j<n; j++)
        {
            c[i][j]=0;
            for(k=0; k<n; k++)
                c[i][j] = c[i][j] + a[i][k]*b[k][j];
        }
    printf("\nProduct matrix\n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<n; j++)
            printf("%d\t",c[i][j]); printf("\n");
    }
}
```

MULTI - DIMENSIONAL ARRAY

Multidimensional arrays are defined in much the same manner as one dimensional arrays, except that a separate pair of square brackets is required for each subscript. Thus, a two dimensional array will require two pairs of square brackets. Three dimensional arrays will require three pairs of brackets and so on.

Definition
Multidimensional array : Collection of elements, all of the same type and same name, ordered on N dimensions ($N \geq 1$). Each element is accessed by N indices, each of which represents the element's position within that dimension.

The general form of the multidimensional array is :

```
type array_name[s1][s2][s3] .....[sm];
```

where s_i is the size of the i^{th} dimension.

Example 14 : Consider a series of department stores (10 stores). Monthly sales figures must be kept for each item by store. There are three important information about each item : the *month* in which it was sold, the *store* from which it was purchased, and the *item* number. We can define an array as follows :

```
#define NUM_ITEMS    7
#define NUM_STORES 10
int    sales[ NUM_STORES ] [ 12 ][ NUM_ITEMS ];
```

sales is a three-dimensional array declared with 10 *stores*, 12 *months* and 7 *items*. This array contain 840 ($10 \times 12 \times 7$) integer type elements. A graphical representation of the **sales** array is shown in Figure.

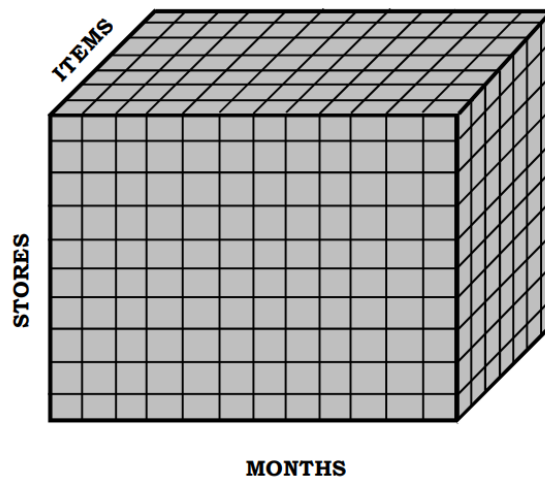


Figure : Graphical representation of sales array

If we want to process the data in the array, we must use subarray processing. The following program segment sums and prints the total number of each item sold this year to date by all stores.

```
for ( item = 0; item < NUM_ITEMS; item++)
{
    NUMBER_SOLD = 0;
    for( store = 0; store < NUM_STORES; store++)
```

```

        for (month = 0; month < CURRENT_MONTH; month++)
            NUMBER_SOLD=NUMBER_SOLD + sales[store][month][item];
        printf("Item %d Sales to date = %d",item,NUMBER_SOLD);
    }

```

Because *item* controls the outer for loop, we are summing each item's sales by *month* and *store*.

Example 15 : Consider the following four-dimensional array declaration

```
Int p[2][3][3][5];
```

p is a four-dimensional array which contains 90 ($2*3*3*5$) integer type elements.

STRINGS AND STRINGS OPERATIONS

INTRODUCTION

Historically, computers were first used for processing numerical data. Today, computers are frequently used for processing nonnumerical data, called character data. This chapter discusses how such data is stored and processed by the computer.

In computer terminology, the term "string" can be defined as a sequence of characters rather than the term "word". Because "word" has different meaning in computer science. For this reason, we use the expression "string processing" or "string manipulation" instead of the

Definition
String: A string is a sequence of characters.

expression "word processing".

String manipulation is often a large part of any programming task, but by creating a library of string manipulation routines, you can greatly decrease the development of many applications. C is one of the few programming languages that provides the programmer with the internal representation of character strings.

STRING MANIPULATION

Character string in C are stored as one-dimensional character array; i.e., successive characters are stored in contiguous memory locations. A string is defined as a character array that is terminated by a null character '\0'. To create a string variable, you must allocate enough space for the number of characters in the string and the NULL character ('\0') that is appended to all string variables and constants.

Example 1 : Consider the following string declaration:

```
char name[12];
```

When the string "Programming" is assigned to a string variable **name**, the characters are stored in the memory as follows:

```
char name [12] = "Programming";
```

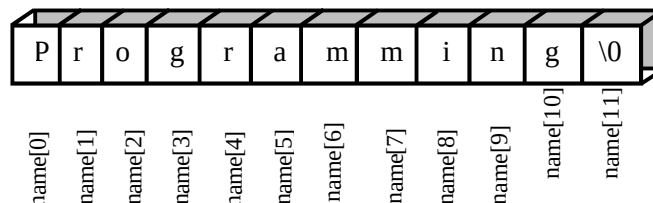


Figure 1 Memory representation of a string

Individual character of a string can be easily accessed by the index. The index 0 refers to the first character, the index 1 refers to the second character, index 2 to the third, and so forth

DECLARATION OF STRING ARRAYS

Like any other variable, string can also be declared. A string is declared as an array of character type. We declare a string of an array by giving the **char** type, its name, and the number of elements. The general form of array declaration is:

Syntax : `char    variable_name[ size ];`

where ***variable_name*** is a string variable and
size is the number of characters in the string.

Example 2: Consider the following declaration

```
char    name [20];
```

reserves a block of 20 bytes (locations) to store the elements of ***name***. Each element can hold a character and its contents can be accessed by its subscript number.

STRING ARRAY INITIALIZATION

Once you declare an array, you can enter its value using **scanf ()** or **gets ()**, you can also assign the array initial values when you declare it.

Example 3: Consider the following string initialization:

Case 1 : `char name [9] = "Computer";`

This creates nine elements with the following values:

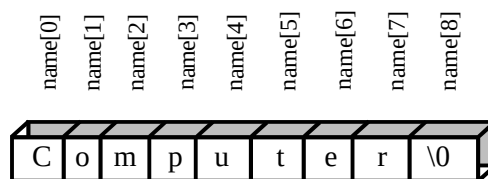


Figure 6.2 Memory representation

The above initialization can also be achieved with either

Case 2 : `char name [ ] = "Computer"`

OR

Case 3 : `char name [9] = {'C','o','m','p','u','t','e','r','\0'};`

/* declare and initialize name [0] = 'C', name [1] = 'o' */

In the above example, the sequence of constants, with commas as dividers, is enclosed in curly braces. The resulting object is called an **initializer**. Each constant in the initializer is assigned in turn to the elements in the array. If you have fewer constants than array elements, the extra array elements are set to Zero.

In the first case, the string terminator ('\0') gets attached automatically since the string has been assigned. In the third case, individual elements are moved into the array, and therefore it is essential that the string terminator be specified explicitly.

BUILDING STRINGS FROM THE INPUT

We often want to build strings from a program's input instead of creating them at compile time. When creating character strings, however, we must append the NULL character to the end of the string, since the NULL character is appended to all of the strings.

To read a string, we use a standard I/O library function **scanf ()** function. **scanf ()** reads formatted input from the standard input, which is usually the keyboard. We can read values into a string with **scanf ()** function using **%s** formatting code:

```
scanf ("%s", name);
```

There is a problem with **scanf ()** function while reading a string data. Consider the following program:

Program 1: A C program to read a string using **scanf ()** function.

```
#include <stdio.h> main( )
{
    char    name[20];
    printf("Enter a string\n");
    scanf("%s",name);
    printf("\nEntered string was %s",name);
}
```

Output 1	Enter a string Srinagar Entered string was Srinagar
Output 2	Enter a string Sri nagar Entered string was Sri

In the second output, the input string was **Sri nagar** but the output result is **Sri**, this is because of the blank space. The **scanf()** function works through the control string, whenever it encounters a field specifications, such as %d, %s, %i, it skips space characters, such as blanks, tabs and line boundaries (while spaces). To avoid such error, C provides another function called **gets()**. The **gets()** function accepts an entire line of input from the standard input device, appends a NULL character '\0' and hands it to the program. The string can be of any length. The newline character '\n' signals the end of the string.

Program 2 : A C program to read a string using **gets()** function.

```
#include <stdio.h>
main()
{
    char    name[20];
    printf("Enter a string\n");
    gets(name);
    printf("\nEntered string was %s", name);
}
```

Output 1	Enter a string Srinagar Entered string was Srinagar
Output 2	Enter a string Sri nagar Entered string was Sri nagar

Another way of reading a string is given in **program 3**. In this program we read characters using **getchar()** function, the program should stop when the character newline '\n' is encountered.

Program 3: Write a C program to read a text, program should stop when the character newline ('\n') is encountered.

```
#include <stdio.h>
main()
{
    char    name[80];
    int i = 0;
    puts("Enter a text");
    while((ch=getchar())!='\n')
    {
        name[i]=ch;
        i++;
    }
    name[i]='\0';          /*    NULL added to end of the string    */
    puts("Input string is");
    puts(name);
}
```

OUTPUT Enter a text C is a powerful programming language Input string is C is a powerful programming language
--

PRINTING A STRING

puts() and **printf()** functions are used to display a string on the standard output device. If we use **printf()** function, the default field widths selected is as described by the designer of the particular C implementation. These default field widths may be adequate for some applications, we will frequently need to control the placement of values on output lines carefully. Generally **printf()** function has the following format:

Syntax `printf("format control string",value_1,value_2,value_3,.....);`

C allows the programmer to explicitly specify field widths for each output data. The % symbol in a *format control* expression may be followed by an optional format modifier, this may specify the number of spaces in which the data must be output, and whether the output should be left or right justified within these spaces.

Example 4 : To output the following string:

Computer Science

right justified in a field of width 25 spaces, use the following **printf()** function.

```
printf("%25s","Computer Science");
```

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24

						C	o	m	p	u	t	e	r		S	c	i	e	n	c	e
--	--	--	--	--	--	---	---	---	---	---	---	---	---	--	---	---	---	---	---	---	---

Here the format control string is "%25s", the format modifier is 25, and the format conversion character is **s**.

A minus sign with the format modifier causes *left* justified printing with the output field.

Example 5: Consider the following program segment.

```
char str1[]="Computer",      str2[ ]="Science";
printf("%-15s %-12s",str1,str2);
```

These statements would produce the following output:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26

C	o	m	p	u	t	e	r					S	c	i	e	n	c	e				
---	---	---	---	---	---	---	---	--	--	--	--	---	---	---	---	---	---	---	--	--	--	--

The **w.d** format modifier with the **s** format conversion character prints the first **d** elements of a string argument right justified in a field of width **w**. The remaining elements of a string are not printed. A preceding minus sign forces left justification.

Example 6.6 : The output of

<pre>printf("%8.5s","Computer");</pre>	<pre>printf("-8.5s","Computer");</pre>																																
<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td></tr><tr><td></td><td></td><td></td><td>C</td><td>o</td><td>m</td><td>p</td><td>u</td></tr></table>	0	1	2	3	4	5	6	7				C	o	m	p	u	<table border="1"><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td></tr><tr><td>C</td><td>o</td><td>m</td><td>p</td><td>u</td><td></td><td></td><td></td></tr></table>	0	1	2	3	4	5	6	7	C	o	m	p	u			
0	1	2	3	4	5	6	7																										
			C	o	m	p	u																										
0	1	2	3	4	5	6	7																										
C	o	m	p	u																													

Example 7 : To print the '%' character and ' ' character C uses the combination of %% and \ respectively. Because a format expression in the format control string being with the % sign, the % character has syntactical significance within a control string.

```
printf("Student\'s average marks is=%.2f%%",average_marks);
```

If the value of average_marks=75.33, the above **printf()** function produces the output:

Student's average marks is = 75.33%

STRING OPERATIONS

C recognizes that string is a different class of array by letting us input and output the array as a unit, and are terminated by NULL character. Fortunately, the C library supports a large number of string handling functions that can be used to carry out many of the string manipulations such as

- Length (number of characters in a string)
- Concatenation (Adding two or more strings)
- Comparing two strings
- Substring (Extract substring from the given string)
- Copy (copies one string over another)

NOTE



To use all the string operations in C you must include **string.h** library header file in the program.

strlen() FUNCTION

This function counts and returns the number of characters in a string. The length does not include the NULL character.

Syntax

```
n = strlen( string );
```

where **n** is an integer variable which receives the value of the length of the **string**.

Example 8 :

```
length=strlen("Bangalore");
```

The function will assign number of characters(9) in the string("Bangalore") to an integer variable **length**.

Program 4: Write a C program to find the length of a string using **strlen()** function.

```
#include<stdio.h>
#include<string.h>
main()
{
    char    name[100];
    int     length;
    printf("Enter a string\n");
    gets(name);
    length=strlen(name);
    printf("\nNumber of characters in the string is %d",length);
}
```

OUTPUT

```
Enter a string Engineering
College
Number of characters in the string is 19
```

Program 5 : Write a C program to find the length of a string without using **strlen()** function.

```
#include <stdio.h>
main()
{
    char name[100];
    int i=0,length;
    printf("Enter a string\n");
    gets(name);
    while(name[i]!='\0')
        i++;
    length=i+1;
    printf("\nNumber of characters in the string is %d",length);
}
```

OUTPUT

```
Enter a string
Pre-university Board
Number of characters in the string is 20
```

Program 6: Write a C program to find the frequency of occurrence of a specified character.

```
#include<stdio.h>
#include<string.h>
main()
{
    char    text[100];
    int     i,length,count=0;
    printf("Enter astring\n");
    gets(text);
    puts("Enter a character to be searched");
    scanf("%c",&ch);
    length=strlen(text);          /*    Length ofthe string    */
    for(i=0;i<length;i++)
        if( text[i]==ch)
            count++;
    if(count)
        printf("\n%c occurs %d times",ch,count);
    else
        printf("\n%c does not exist in the text",ch);
}
```

Output 1

Enter a string
Sri Bhagawan Mahaveer Jain College
Enter a character to be searched a
a occurs 6 times

Output 2

Enter a string
Bangalore
Enter a character to be searched p
p does not exist in the text

strcpy() FUNCTION

C does not allow you to assign characters to a string directly, as in the statement:

```
char name[30];  
name="Shanthamani";
```

Instead, use the **strcpy()** function found in most compilers. The syntax of the function is illustrated below:

Syntax

```
strcpy( string1, string2);
```

and assign the contents of **string2** to **string1**. **string2** may be a character array variable or a string constant.

Example 9: **strcpy(name,"Shanthamani");**

In the above example, the characters "Shanthamani" are assigned to the string called **name**.

Program 7: A program to copy one string to another without using library function.

```
#include <stdio.h>  
main()  
{  
    char    source[100],destination[100];  
    int     i=0;  
    printf("Enter a string");  
    gets(source);  
    while(source[i]!='\0')  
    {  
        destination[i]=source[i];  
        i++;  
    }  
    destination[i]='\0';  
    printf("\n%d characters are transferred to destination string",i);  
    printf("\nDestination string is %s",destination);  
}
```

OUTPUT

```
Enter a string Second Year PUC  
15 characters are transferred to destination string  
Destination string is Second Year PUC
```

strcat() FUNCTION

When you combine two strings, you add the characters of one string to the end of another string. The process is called concatenation. The **strcat()** function joins two strings together. It takes the following form:

Syntax

```
strcat( string1, string2);
```

string1 and **string2** are character arrays. When the function **strcat()** is executed, **string2** is appended to **string1**. The string at **string2** remains unchanged.

Example 10 :

```
strcpy( string1,"Sri");
strcpy( string2,"Bhagawan");
printf("%s",strcat(string1,string2));
```

From the above program segment the value of **string1** becomes "Sri Bhagawan". The string at **string2** remains unchanged i.e. "Bhagawan".

Program 8 : Write a C program to concatenate two strings without using **strcat()**.

```
#include <stdio.h>
#include <string.h>
main()
{
    char    str1[100],str2[100];
    int     i,length1,length2;
    printf("Enter the first string\n");
    gets(str1);
    length1=strlen(str1);
    printf("Enter second string\n");
    gets(str2);
    length2=strlen(str2);
    /*      append the content of str1 to str2      */
    for(i=0;i<length2;i++)
        str1[length1+i]=str2[i];
    str1[length1+length2]='\0';
    printf("Concatenated string is %s",str1);
}
```

OUTPUT

```
Enter the first string Kapil
Enter second string Dev
Concatenated string is Kapil Dev
```

strcmp() FUNCTION

In C, you cannot directly compare the value of two strings in a condition like this :
`if(string1==string2)`

Most libraries, however, contain the **strcmp()** function, which returns a zero, if two strings are equal, or a nonzero number if the strings are not the same. The syntax of **strcmp()** is given below

Syntax

```
strcmp(string1,string2);
```

string1 and **string2** may be string variables or string constants. Some compilers return a negative if the **string1** is alphabetically “less than” the second, and a positive number if the first string is greater than the second.

Example 11 : `strcmp("MYSORE","MYSORE")` will return 0 because two strings are equal.

`strcmp("their","there")` will return -9 which is the numeric difference
between ASCII "i" and ASCII "r". (i.e., fourth position)

`strcmp("there","their")` will return 9 which is the numeric difference
between ASCII "r" and ASCII "i".

`strcmp("THE","the")` will return -32 which is the numeric difference
between ASCII "T" and ASCII "t".

strncmpi() FUNCTION

This function is same as **strcmpi()**, which compares two strings, but not case sensitive.

Example 12 : `strncmpi("THE","the")` will returns 0.

strlwr() FUNCTION

This function converts all characters in a string from uppercase to lowercase

Syntax

```
strlwr(string);
```

For example: `strlwr("C PROGRAMMING")` converts "C PROGRAMMING" into
"c programming".

Program 9: A C program to accept a string and converts all characters in a string from uppercase to lowercase without using library function.

```
#include <stdio.h>

main()
{
    char   str[100];
    int    i=0;
```

```

printf("Enter a string ");
gets(str);
while(str[i]!='\0')
{
    if(str[i]>='A' && str[i]<='Z')
        str[i]=str[i]+32;
    i++;
}
printf("\nConverted string in lowercase is %s",str);
}

```

OUTPUT

Enter a string C ProGRamming
Converted string in lowercase is c programming

strrev () FUNCTION

This function reverses the characters in a string

Syntax

```
strrev(string);
```

For example, **strrev**("program") reverses the characters in a string into "margorp".

Program10: A C program to accept a string and print it in the reverse order without using library function.

```

#include <stdio.h>
#include <string.h>

main()
{
    char    str[100],revstr[100];
    int     i,length;
    printf("Enter a string");
    gets(str);
    length=strlen(str);          /*    Length of the string    */
    for(i=0;i<length;i++)
        revstr[i]=str[length-i-1];
    revstr[length]='\0';
    printf("\nReversed string is %s",revstr);
}

```

OUTPUT

Enter a string ELECTRONICS Reversed
string is SCINORTCELE

Program 11: A C program to check whether a given word is palindrome or not (A palindrome is defined as a word read from left right and right to left give the same word) Examples : MADAM, MALAYALAM, LEVEL, RADAR

```
#include <stdio.h>
#include <string.h>
main()
{
    char str[10],str2[10];
    printf("Enter a string ");
    scanf("%s",str1);
    strcpy(str2,str1);/* copy the content of str1 to str2 */
    strrev(str2);      /*reverse the content of str2 */

    if( strcmp(str1,str2)==0)
        printf("%s is a palindrome",str1);
    else
        printf("%s is not a palindrome", str1);
}
```

OUTPUT

```
Enter a string MALAYALAM
MALAYALAM is a palindrome
Enter a string BANGALORE
BANGALORE is not a palindrome
```

strupr() FUNCTION

This function converts all characters in a string from lowercase to uppercase.

Syntax

```
strupr( string );
```

For example `strupr("c programming")` converts "c programming" into "C PROGRAMMING".

Program 12: A C program to convert all characters in a string from lowercase to uppercase.

```
#include <stdio.h>

main()
{
    char str[100];
    int i=0;
    printf("Enter a string ");
    gets(str);
```



```
while(str[i]!='\0')
{
    if(str[i]>='a'&&str[i]<='z')
        str[i]=str[i]-32;
    i++;
}
printf("\nConverted string in Uppercase is %s",str);
}
```

OUTPUT

Enter a string C ProGRamming

Converted string in lowercase is C PROGRAMMING

STRING HANDLING FUNCTIONS

The string header file (*string.h*) declares one type and several functions, and defines several macros all of which are useful for manipulating strings. Following are the most commonly used string handling functions.

FUNCTION	DESCRIPTION
strcat	Adds the characters of one string to another.
strchr	Returns the position of a specified character in the string.
strcmp	Compares two strings.
strcmpi	Compare two strings; not case sensitive.
strcpy	Copies one string or string literal, to another.
strcspn	Returns the position of a character in the string from a specified character set.
strlen	Calculates the string length.
strlwr	Converts a string to lowercase.
strncat	Appends specified characters from one string to another.
strncmp	Compares specified characters of two strings.
strncpy	Copies specified characters from one string to another.
strnset	Changes specified characters in a string to another character.
strrev	Reverses the characters in a string.
strstr	Finds one string within another.
strupr	Converts a string to uppercase.

ARRAY OF STRINGS

An array of strings is a two-dimensional character array. The size of first index (rows) determines the number of strings and the size of second index (columns) determines maximum length of each string.

Example 13 : Consider the following array declaration

```
char name[8][20];
```

declares an array of 8 strings, each of which can hold maximum of 20 characters. An array of strings appears in memory as shown in Figure.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
name [0]	D	R	U	V	A	\0														
name [1]	G	U	L	N	D	A	R	\0												
name [2]	S	H	I	V	A	J	I	\0												
name [3]	C	H	I	N	T	A	N	\0												
name [4]	M	Y	S	O	R	E	\0													
name [5]	B	A	N	G	A	L	O	R	E	\0										
name [6]	K	A	R	A	N	\0														
name [7]	H	A	R	I	S	H	\0													

Figure: Array of Strings

It is very easy to access an individual string by just specifying the first index, an individual string can be accessed. For example, in above figure, name[1] would give you "GULBARGA"; name[5] would give you "BANGALORE". Following example program uses an array of strings.

Program 13: A C program to arrange N names in alphabetical order.

```
#include<stdio.h>
#include <string.h>
main()
{
    char name[10][20];
    char tempname[20];
    int n,i,j;
    printf("Enter number of names ");
    scanf("%d",&n);
    printf("Enterlistofnames\n");
    for(i=0;i<n;i++)
        gets(name[i]);
    for(i=1;i<n;i++)
        for(j=0;j<n-i;j++)
            if(strcmp(name[j],name[j+1]) >0)
            {
                strcpy(tempname,name[j]);
                strcpy(name[j],name[j+1]);
                strcpy(name[j+1],tempname);
            }
    printf("\nSorted names");
    for(i=0;i<n;i++)
        printf("\n%s",name[i]);
}
```

OUTPUT

```
Enter number of names      5
Enter list of names
TANGALORE
BANGALORE
MYSORE
JAYANAGAR
SRINAGAR

Sorted names
BANGALORE
JAYANAGAR MYSORE
SRINAGAR
TANGALORE
```

OPERATION ON CHARACTERS (ctype.h)

Sometimes, it may be required to analyse a character type such as alphanumeric, digits, control characters etc. These facilities are declared in the header file **ctype.h**.

Character manipulation functions are of two types: *classification* and *conversion*. The classification functions, which begin with the prefix *is*, are used for determining the type of a character, e.g., *isalpha*. The conversion functions convert a lower-case letter to upper case and vice versa.

classification character functions			
Function	Description	Syntax	Examples
isalnum()	Tests whether a character is an alphabet or a number	isalnum(c)	isalnum('+')=0 isalnum('a')=1 isalnum('0')=1
isalpha()	Character classification that returns non-zero if c is a letter(A-Z or a-z)	isalpha(c)	isalpha('0')=0 isalpha('a')=1
isascii()	Tests whether a character is an ascii(0 to 127) character.	isascii(c)	isascii('a')=1 isascii('!')=0
isctrl()	Tests whether a character is a control character.	isctrl(c)	
isdigit()	Character classification that returns non-zero value, if c is a digit ('0'-'9')	isdigit(c)	isdigit('6')=1 isdigit('a')=0
islower()	Character classification that returns non-zero value if c is a lowercase.	islower(c)	islower('a')=1 islower('A')=0 islower('0')=0
isprint()	Character classification that returns value if c is a printable character (ascii 32 to 126) and nonzero otherwise.	isprint()	isprint()=1 isprint()=0
isspace()	Character classification that returns non-zero value if c is a space, tab, carriage return, newline, vertical tabor formfeed.	isspace()	isspace('\n')=1 isspace('a')=0
isupper()	Character classification that returns non-zero value if c is an uppercase letter ('A'-'Z')	isupper(c)	isupper('M')=1 isupper('n')=0
isxdigit()	Character classification that returns non-zero value if c is a hexadecimal digit (0-9,A-F,a-f)	isxdigit(c)	isxdigit('p')=0 isxdigit('0')=1 isxdigit('E')=1

Conversion character functions			
Function	Description	Syntax	Examples
toascii()	Translates character to ASCII format.	toascii(c)	toascii('0')=48 toascii('Z')=90
tolower()	Translates character to lowercase.	tolower(c)	tolower('S')=s tolower('M')=m
toupper()	Translates character to uppercase.	toupper(c)	toupper('g')=G
_tolower()	Translates character to lowercase.	_tolower(c)	_tolower('S')=s _tolower('M')=m
_toupper()	Translates character to uppercase.	_toupper(c)	_toupper('g')=G

Program 14 : Input a string and find the number of numeric characters, lowercase alphabetic characters, uppercase alphabetic characters and any characters other than the above.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
main()
{
    char str[100];
    int upalpha=0,lowalpha=0,digit=0,special=0,length,i;
    printf("Enter a string ");
    gets(str);
    length=strlen(str);          /*Find length of the string */
    for(i=0;i<length;i++)
        if(isupper(str[i])) upalpha++;
        else if(islower(str[i])) lowalpha++;
        else if(isdigit(str[i])) digit++;
        else special++;
    printf("\nNumber of uppercase=%d",upalpha);
    printf("\nNumber of lowercase=%d",lowalpha);
    printf("\nNumber of digit = %d",digit);
    printf("\nNumber of special=%d",special);
}
```

OUTPUT

```
Enter a string SUM of 12 and 13 IS =25
Number of uppercase = 5
Number of lowercase = 5
Number of digit = 6
Number of special = 6
```

Program 15 : Input a string, find the number of vowels appearing in the string.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
main()
{
    char    str[100];
    int     i,length,count=0;
    printf("Enter a string:");
    gets(str);
    length=strlen(str);          /*Find length of the string    */
    for(i=0;i<length;i++)
        switch(toupper(str[i]))
        {
            case 'A'      :
            case 'E'      :
            case 'I'      :
            case 'O'      :
            case 'U'      :      count++;
        }
    printf("%d vowels occur in the string %s",count,str);
}
```

OUTPUT

Enter a string: Bangalore is the garden city of India.
13 vowels occur in the string Bangalore is the garden city of India

TYPE CONVERSIONS

C can perform conversions between different data types, but a programmer should take care with these conversions. There are two types of type conversions.

1. Implicit type conversion
2. Explicit type conversion

The process of converting one predefined type into another is called type conversion.

Implicit type conversion:

When we perform arithmetic manipulation on operands belonging to different data types, they may undergo type conversion automatically before evaluating the final value. This conversion is performed by the compiler without programmer's intervention. This conversion is applied generally whenever different data types are intermixed in an expression. When variables of different types appear in expressions, the following rules apply.

Operand 1	Operand 2	Result
float char	int int	Converts int to float returns float . Treats both as int and returns int . This is a dangerous procedure
float	double	Converts float to double and returns double . An assignment may change the result back to float .
int int unsigned	short long int	For example, float= float operator double Converts short to int Converts int to long Converts int to unsigned if no sign is present

Other conversions occur in assignment statements.

float=int;	Converts int to float	float a; a=5; value of a=5.000000
int=float;	Truncates the fractional part of float.	int a; a=45.67; value of a=45
int= char;	Assigns the ASCII value of char to int.	int ascii; ascii='A'; value of ascii= 65
char=int;	Depends on the value on int. If it is within the values allowed for char.If not, it is compiler independent.	char ch; ch=66; value of ch ='B'
double=float; float=double;	Converts float to double. Rounds off double as needed.	

Explicit conversion (Type Casting):

An explicit type conversion is user-defined that forces an expression to be of specific type. Variables may be converted explicitly by type casting. The general form is

(data_type) expression;

Where data_type is any allowed data type like int,char, float or double, and expression includes either constant, variable or expression.

Example:

```
int x;
.....
(float)x;
```

The variable x is converted from int to float.

Example: Type casting can be done within the assignment statement.

```
int sum, count;
float average;
average=(float)sum / count;
```

This assignment statement causes sum to be converted to type float before the division takes place. Casting is frequently used when integers are passed as arguments to mathematical functions.

getch() function:

The getch() is a predefined non-standard function that is defined in conio.h header file. It is mostly used by the Dev C/C++, MS- DOS's compilers like Turbo C to hold the screen until the user passes a single value to exit from the console screen. It can also be used to read a single byte character or string from the keyboard and then print. It does not hold any parameters. It has no buffer area to store the input character in a program.

We use a getch() function in a C/ C++ program to hold the output screen for some time until the user passes a key from the keyboard to exit the console screen.

Syntax: int getch(void);

The getch() function does not accept any parameter from the user. It returns the ASCII value of the key pressed by the user as an input.

Example: Write a simple program to hold the console screen until the user gives a key to exit.

```
#include <stdio.h>
#include <conio.h>
int main()
{
    printf("Enter a key to exit the console screen.\n");
    getch();
    return 0;
}
```

Example: Write a program to use the getch() function to accept the string from the user.

```
#include <stdio.h>
#include <conio.h>
int main() {
    char ch[6] = {0}; int x;
    for ( x = 0; x < 5; x++) {
        ch[x] = getch(); // getch() function to take input
    }
    printf("Received 5 character Input: %s\n", ch);
    return 0;
}
```


putch() function:

The putch() function is used for printing character to a screen at current cursor location. It is unformatted character output functions. It is defined in header file conio.h.

Syntax:

```
putch(character);  
    or  
putch(character_variable);
```

Example: Displaying Character Using putch()

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    char ch;  
    printf("Press any character: ");  
    ch = getch();  
    printf("\nPressed character is:");  
    putch(ch);  
    getch(); /* Holding output */  
}
```